Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

# PROYECTO 3 – SISTEMA DE AUTORIZACIÓN Y CONTROL DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS (DTE)

**202500708 – David Antonio Meza Silva**

**Resumen**

Este ensayo describe el diseño e implementación de un simulador del sistema de autorización de Documentos Tributarios Electrónicos (DTE) de la Superintendencia de Administración Tributaria de Guatemala. El sistema recibe lotes de solicitudes en formato JSON, valida cada factura, asigna un número de autorización correlativo diario y administra el Registro Tributario Unificado (RTU).

La solución se compone de dos programas comunicados por HTTP: un servicio backend en ASP.NET Core Web API, que almacena la información en estructuras enlazadas propias y la persiste en archivos JSON, y un cliente web en Razor Pages que permite enviar mensajes, consultar reportes, visualizar gráficas y exportarlos a PDF.

Entre los algoritmos principales se encuentran la validación de NIT mediante Módulo 11, la extracción de fecha y hora con expresiones regulares, el control de referencias únicas y la verificación del IVA. Se concluye que separar el servicio del cliente facilita probar el sistema con herramientas externas, y que las estructuras propias bastan para el volumen de datos del problema, aunque su búsqueda lineal limitaría el crecimiento del RTU.

**Palabras Clave:**

Documento Tributario Electrónico (DTE); Módulo 11; API REST; Tipos de Datos Abstractos; Registro Tributario Unificado (RTU).

**Abstract**

This essay describes the design and implementation of a simulator of the Electronic Tax Document (DTE) authorization system of Guatemala's Tax Administration Authority. The system receives batches of requests in JSON format, validates each invoice, assigns a daily sequential authorization number, and maintains the Unified Tax Register (RTU).

The solution consists of two programs that communicate over HTTP: a backend service built with ASP.NET Core Web API, which stores data in custom linked structures and persists it in JSON files, and a Razor Pages web client that sends messages, queries reports, displays charts, and exports them to PDF. The main algorithms are tax ID validation using Modulo 11, date and time extraction with regular expressions, reference uniqueness control, and VAT verification. The essay concludes that separating the service from the client makes it easier to test the system with external tools, and that the custom structures are sufficient for the problem's data volume, although their linear search would limit the growth of the RTU.

**Keywords:**

Electronic Tax Document (DTE); Modulo 11; REST API; Abstract Data Types; Unified Tax Register (RTU).

## Introducción

En Guatemala, la Superintendencia de Administración Tributaria (SAT) regula la facturación electrónica en línea. Cada documento debe validarse y autorizarse antes de ser aceptado, por lo que se requiere un servicio que aplique reglas formales y aritméticas sobre lotes de facturas.
El enunciado del proyecto plantea construir un servicio HTTP que reciba solicitudes en JSON, verifique los NIT del emisor y del receptor contra el RTU, compruebe la unicidad de la referencia y el cálculo del IVA, y emita un código de autorización correlativo por día (Facultad de Ingeniería, 2026). Los datos deben persistir en archivos.
Este ensayo explica cómo se resolvió, a partir de una pregunta guía: ¿cómo estructurar un servicio y un cliente web desacoplados que validen mensajes JSON, almacenen sus resultados en disco y generen los reportes solicitados?

## Desarrollo del tema

### a. Abstracción del Problema y Programación Orientada a Objetos

El sistema modela el flujo real de autorización: un contribuyente emite una factura, la SAT la valida y, si es correcta, le asigna un código de autorización; de lo contrario, la rechaza e indica el motivo.

Con programación orientada a objetos en C#, este flujo se representó con las siguientes clases (Anexo A):

- **Factura**: modela el DTE en evaluación. Contiene los atributos del JSON de entrada (TIEMPO, REFERENCIA, NIT_EMISOR, NIT_RECEPTOR, VALOR, IVA y TOTAL) y los datos generados al procesarla (FueAprobada, CodigoAutorizacion, MotivoRechazo y la fecha normalizada FechaParseada).
- **ContribuyenteDto:** Representa a un contribuyente inscrito en el RTU, con su NIT normalizado y su nombre.
- **RegistroCorrelativo:** Asocia una fecha con el último correlativo de autorización emitido ese día.
- **TipoError:** Enumera las causas de rechazo: NIT de emisor o receptor inválido (Módulo 11), emisor o receptor inexistente en el RTU, referencia duplicada, e IVA o total mal calculados.
- **SatController:** Controlador REST que recibe las solicitudes HTTP y las delega a los servicios.

La lógica de negocio se concentra en ServicioRtu, ServicioAutorizacion y ServicioReportes, mientras que Repositorio se encarga del acceso a los datos.

### b. Estructuras de datos propias y persistencia

Se implementaron las estructuras Nodo, ListaSimple, Pila y Cola. Aunque el enunciado permite usar las colecciones de C#, se optó por TDA propios para controlar el recorrido y la inserción, y para convertir las listas a arreglos de forma directa al serializar a JSON.

Como cada lote contiene decenas o cientos de registros, se eligió una lista simplemente enlazada con un puntero al último nodo (ultimo). Así, insertar al final cuesta O(1) y no se recorre la lista completa:

```
public void Insertar(object dato)
{
    Nodo nuevo = new Nodo(dato);
    if (primero == null) {
            primero = nuevo;
            ultimo = nuevo;
    } else {
            ultimo!.Siguiente = nuevo;
            ultimo = nuevo;
    }
     cantidad++;
}
```

Repositorio mantiene cuatro listas en memoria: contribuyentes, facturas aprobadas, facturas rechazadas y correlativos diarios. Después de cada actualización del RTU o procesamiento de facturas, el estado se guarda en cuatro archivos JSON (rtu.json, aprobadas.json, rechazadas.json y correlativos.json). Para detectar referencias repetidas sin recorrer la lista de aprobadas, el repositorio mantiene también un HashSet<string> con las referencias ya autorizadas, con verificación en O(1) promedio.

**c. Algoritmos Principales**

El procesamiento de cada solicitud sigue una secuencia de validaciones (Anexo C).

**1. Validación de NIT con Módulo 11**

El NIT tiene de 1 a 20 dígitos seguidos de un dígito verificador (0-9 o K), es decir, entre 2 y 21 caracteres. A cada dígito del cuerpo se le asigna un peso que empieza en 2 en el dígito más cercano al verificador y aumenta hacia la izquierda (Anexo B):

$$S = \sum_{i=1}^{n} d_i \, (i + 1)$$

Donde:

di = i-ésimo dígito del cuerpo del NIT, contando desde la derecha y sin incluir el verificador
n = cantidad de dígitos del cuerpo
S = suma ponderada

El dígito esperado se obtiene con:

V = (11 - S mod 11) mod 11

donde V es el dígito verificador esperado. Si V = 10, el verificador debe ser K; en otro caso, debe ser igual a V. Por ejemplo, para el NIT 7378106 el cuerpo es 737810, S = 137, 137 mod 11 = 5 y V = 6, que coincide con el último dígito.

**2. Extracción de fecha y hora con expresiones regulares**

El campo TIEMPO incluye texto adicional (por ejemplo, "Guatemala, 15/01/2025 15:25 hrs."). La clase ExtractorTiempo aplica la expresión regular:

(\d{1,2})/(\d{1,2})/(\d{4})\s+(\d{1,2}):(\d{2})

que captura solo la fecha y la hora e ignora el resto del texto sin considerarlo un error.

**3. Validación de la factura**

Para que una factura sea aprobada debe cumplir lo siguiente:

- Ambos NIT tienen formato válido según el Módulo 11.

- Ambos NIT existen en el RTU. Si faltan los dos, se incrementan los contadores RechazadasEmisor y RechazadasReceptor, y se registran ambas causas para que el reporte de errores cuente los dos fallos.
- La referencia no fue utilizada antes.
- El VALOR es mayor que cero y el IVA coincide con el 12 % del valor:
  IVA = redondear(VALOR * 0.12, 2)

El sistema acepta tanto el redondeo comercial (MidpointRounding.AwayFromZero) como el bancario (ToEven), para no rechazar facturas por diferencias de redondeo entre herramientas.

- El TOTAL es igual al valor más el IVA:

TOTAL = VALOR + IVA

**4. Asignación del código de autorización**

Al aprobar una factura se genera el código con formato yyyymmdd########. Se forma con la fecha en que se emite la autorización (DateTime.Now) y un correlativo de ocho dígitos rellenado con ceros a la izquierda (por ejemplo, 2026102000000001), que inicia en 1 cada día.

La Tabla I resume el costo de las operaciones principales. El procesamiento de un lote es la operación más costosa, porque cada factura se busca en el RTU.

***Tabla I**. Costo de las operaciones principales.*

| Operación | Costo | Descripción |
|---|---|---|
| Inserción Lista Simple | O(1) | Inserción al final mediante el puntero ultimo |
| Validación Módulo 11 | O(L) | Recorrido del NIT de longitud (2 <= L <= 21). |
| Búsqueda de contribuyente | O(c) | Búsqueda secuencial entre c contribuyentes |
| Verificación de referencia | O(1) | Consulta en el HashSet de referencias aprobadas. |
| Generación de correlativo | O(d) | Búsqueda del registro del día entre d fechas. |
| Procesamiento de un lote de *f* facturas | O(f * c) | Cada factura se compara contra el RTU. |
| Ordenamiento de reportes | O(k log k) | Array.Sort sobre las k fechas del reporte |

***Fuente**. Elaboración Propia.*

**d. Arquitectura y experiencia de usuario**

El sistema se divide en dos programas independientes que se comunican por HTTP (Anexo D):

- **Servicio 2 (backend):** Web API de ASP.NET Core. Expone los endpoints /Procesar, /ConsultaDatos, /ResumenIva, /ResumenRango, /ReporteErrores e /Inicializar, cada uno disponible también con el prefijo /api/. Lee JSON sin distinguir mayúsculas de minúsculas y acepta fechas en formato dd/MM/yyyy e ISO yyyy-MM-dd, por lo que puede consumirse desde el cliente web y desde herramientas como Postman.

- **Programa 1 (frontend):** cliente en Razor Pages que simula la aplicación principal. Permite enviar uno o varios archivos .json a la

vez y muestra la respuesta formateada. Ofrece cuatro consultas:

1. *Consultar contribuyente:* filtra por NIT y rango de fechas, con el detalle diario y el total acumulado.
2. *Resumen de IVA:* muestra el IVA emitido y recibido por NIT en una fecha, con una gráfica comparativa.
3. *Resumen por rango:* muestra el valor autorizado por fecha, con IVA o sin IVA, en una tabla y una gráfica.
4. *Diagnóstico de errores:* presenta por fecha las facturas recibidas, los siete tipos de error, las facturas sin error y la cantidad de emisores y receptores distintos.

Las gráficas usan Chart.js alojado en el propio servidor, por lo que el sistema no depende de recursos externos y funciona sin conexión a Internet. Cada consulta incluye estilos de impresión (@media print) con el membrete de la SAT, de modo que el navegador puede guardar el reporte, con sus tablas y gráficas, como archivo PDF.

**Conclusiones**

Separar el sistema en un servicio Web API y un cliente Razor Pages comunicados con JSON sobre HTTP permitió mantener la lógica tributaria y el cálculo de correlativos en un solo lugar, accesible desde cualquier cliente, incluido Postman.

La lista enlazada con puntero al último nodo resultó suficiente para el volumen del problema: la inserción es de tiempo constante y las listas se convierten fácilmente a arreglos para guardarlas en JSON. El uso de un HashSet para las referencias evitó que la detección de duplicados se volviera cuadrática en lotes grandes.

La combinación del Módulo 11, las expresiones regulares y la verificación del IVA y del total permite descartar el texto adicional de los mensajes y rechazar las facturas inconsistentes antes de autorizarlas.
El diseño tiene limitaciones. La búsqueda de contribuyentes es lineal, lo que afectaría el tiempo de respuesta si el RTU creciera a millones de registros. Además, el repositorio es una única instancia sin control de concurrencia, y cada lote reescribe completos los archivos JSON. Como trabajo futuro se propone indexar el RTU con una tabla hash o un árbol balanceado, proteger el acceso concurrente (por ejemplo, con lock en la generación de correlativos) y migrar el almacenamiento a una base de datos.

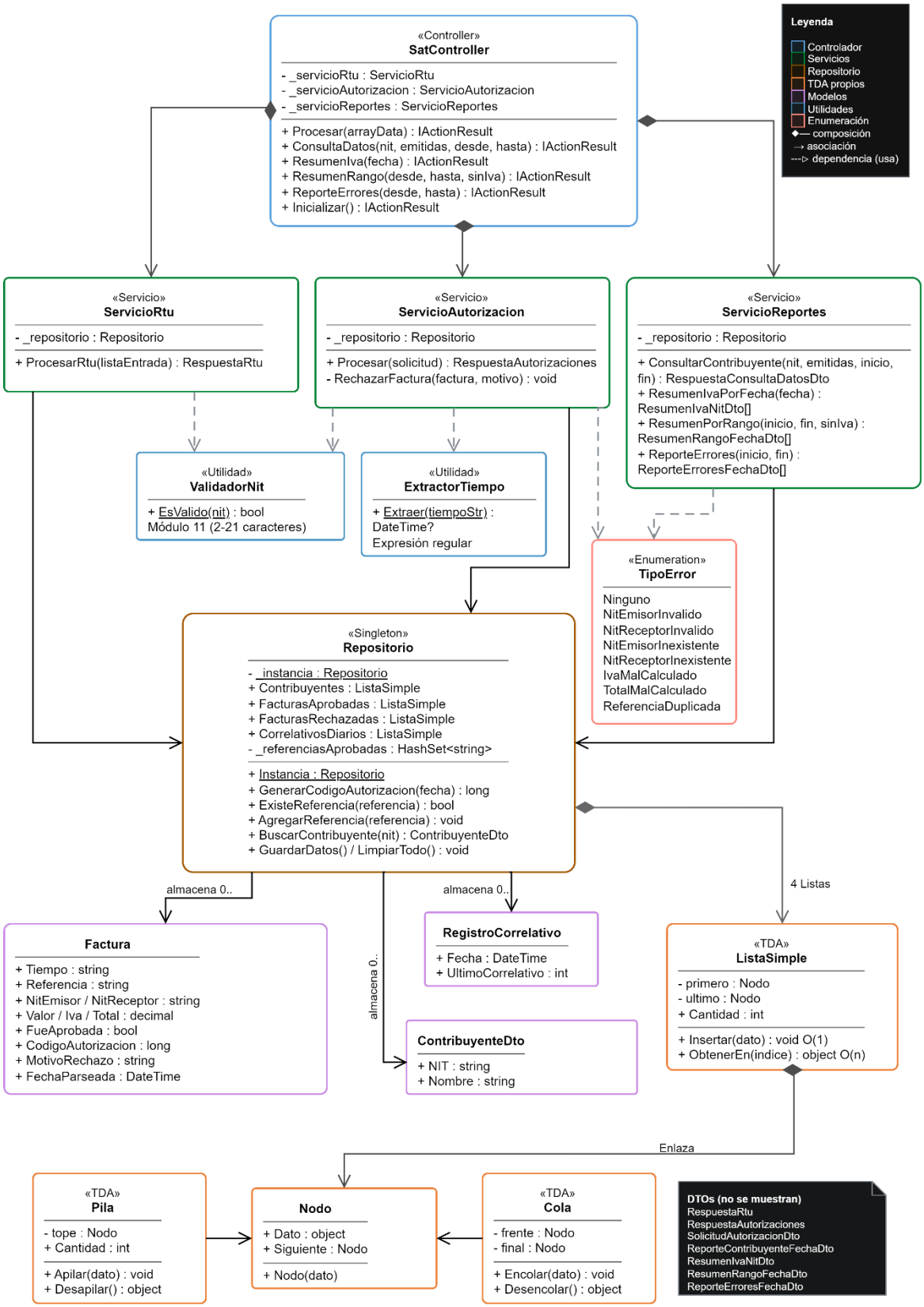
**Referencias Bibliográficas**

- Auxiliatura de Introducción a la Programación y Computación 2. (2026). *Proyecto 3: Sistema De Autorización Y Control De Documentos Tributarios Electrónicos (DTE).* https://proyectos-auxiliatura-ipc2.vercel.app/proyecto3

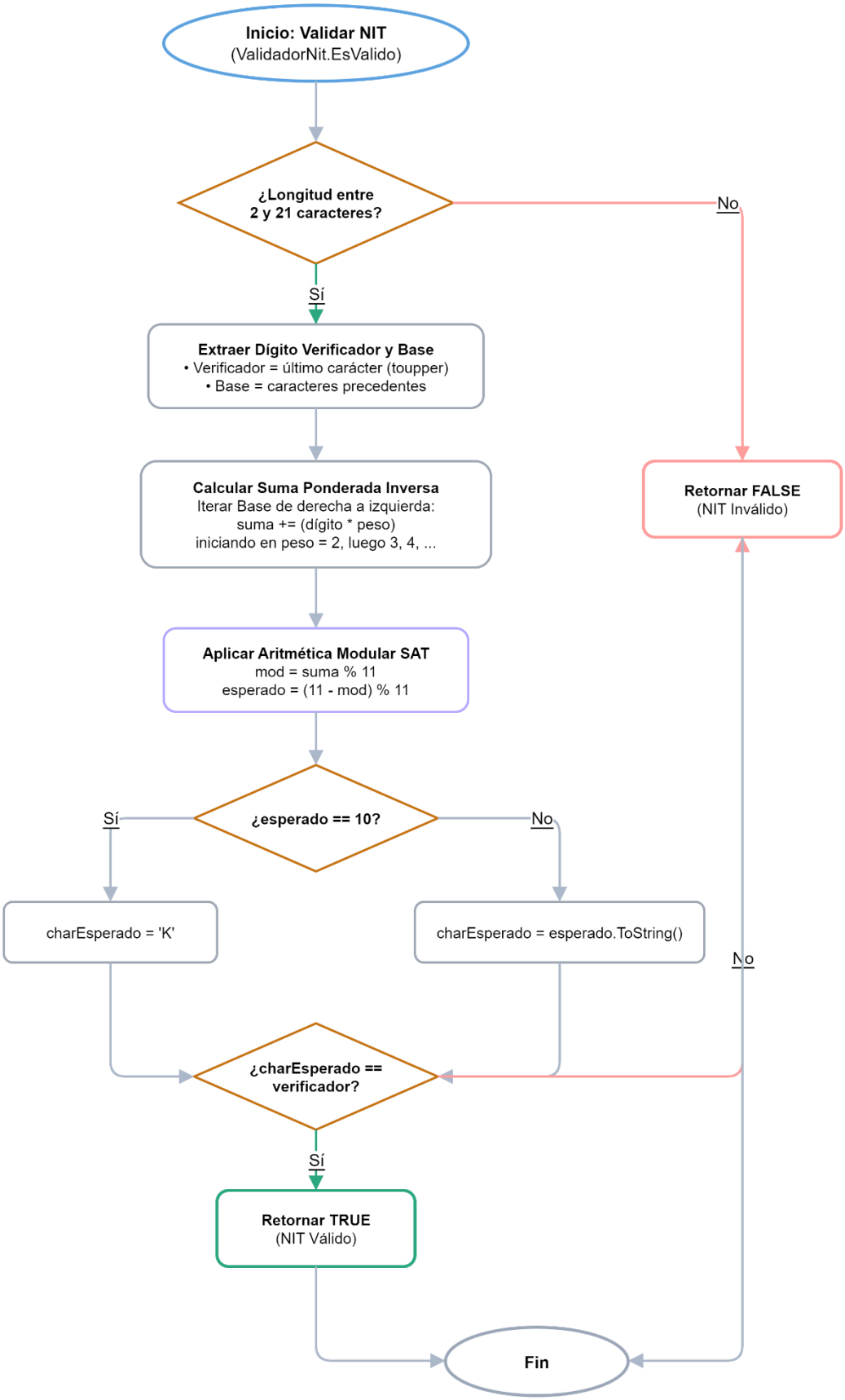
- Facultad de Ingeniería, Universidad de San Carlos de Guatemala. (2026). *Proyecto 3*: *Sistema De Autorización Y Control De Documentos Tributarios Electrónicos (DTE).* Introducción a la Programación y Computación 2.

# Anexos

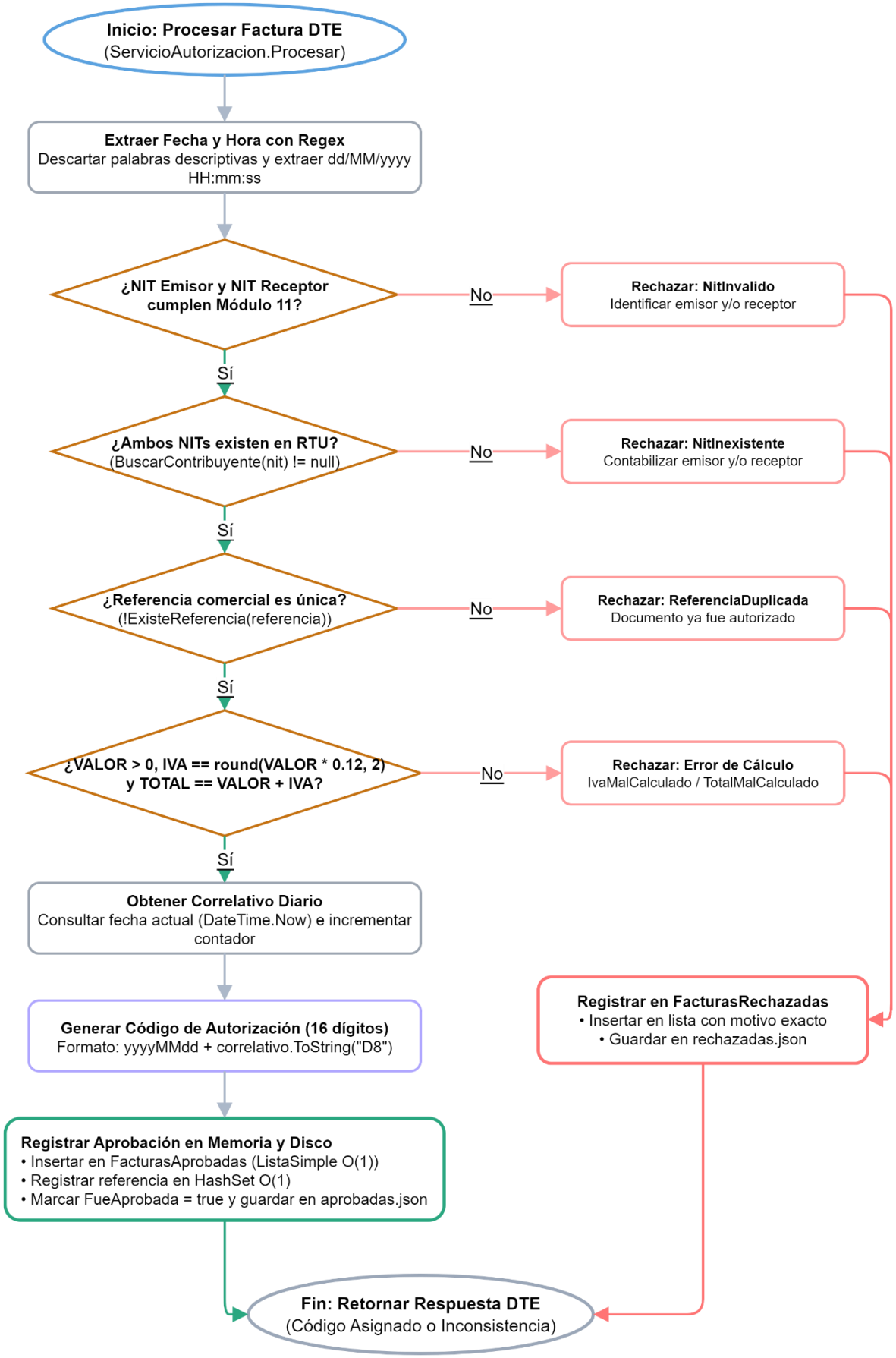
## **Anexo A.** Diagrama de clases.



**Anexo B.** Diagrama de actividad – Algoritmo Módulo 11 para validación de NIT



**Anexo C.** Diagrama de actividad – Flujo de validación y autorización de facturas DTE.



**Anexo D.** Diagrama de despliegue del software desarrollado

.

